FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Alvin Christian S. Panio
Section:	AN43-BSCSSE
Professor:	Crisola Tan
Date:	October 6, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA1)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.

5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()
```

```
# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:

- How agents spread across the grid over time
- Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
  - **More agents** (e.g., 10 or 20)
  - **Smaller/larger grid sizes**
  - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩

Completed updates: the program records every path, prevents agents from entering occupied cells, reports final positions, and compares two parameter scenarios. The complete tested code and outputs are included after the question responses.

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. High agent density increases crowding. With the implemented avoidance rule, empty destinations become scarce, so agents are frequently blocked and remain near a small set of cells. Without avoidance, several agents could occupy the same cell and visible clusters would be common.

2. More steps usually increase exploration. Agents have more opportunities to move farther from their starting positions and visit additional cells. The increase is not unlimited because paths can revisit cells, and boundaries or crowding can cause repeated stay-in-place steps.

3. The agents do not leave the grid. Each proposed coordinate is clamped to the valid range from 0 to grid_size - 1, the grid is non-toroidal, and AgentPy moves agents through grid.move_to. A direction that points beyond an edge is recorded as a stay-in-place step.

4. The model represents movement under uncertainty. Random walks can approximate diffusion or Brownian motion, animal foraging, pedestrian movement, and the spread of information or disease. Additional rules for obstacles, congestion, attraction, or preferred directions make the model closer to a chosen real system.

Observed evidence: Scenario A completed 94% of attempted steps on average, while crowded Scenario B completed only 17%, confirming that high density strongly limits movement under collision avoidance.

LABORATORY RESULTS

The corrected AgentPy model was executed twice with deterministic seed 42. Both runs used bounded four-direction movement, tracked every path, and prevented agents from entering occupied cells.

Metric	Scenario A	Scenario B
Parameters	5 agents, 10x10 grid, 20 steps	20 agents, 5x5 grid, 50 steps
Final occupied cells	5 of 100 (5%)	20 of 25 (80%)
Average successful moves	18.8 of 20 (94%)	8.5 of 50 (17%)
Average blocked/stay steps	1.2 of 20	41.5 of 50
Average unique cells visited	11.60 per agent	5.35 per agent
Observed pattern	Agents moved freely across larger areas.	Crowding restricted most movement.

Comparison: Scenario A visited more cells and completed almost every step. Scenario B placed 20 agents in only 25 cells; the small number of empty destinations caused an average of 41.5 blocked or boundary steps per agent. The paths therefore remained short and local even though the run contained more steps.

SCENARIO A OUTPUT

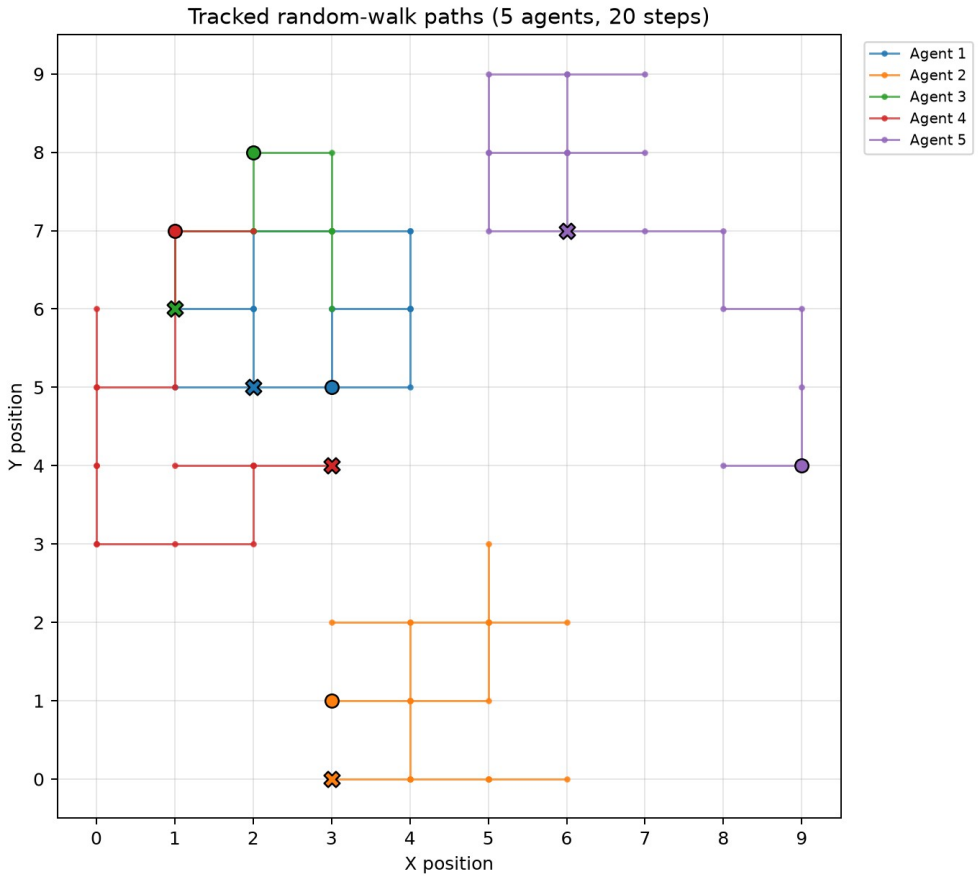


Figure 1. Five agents on a 10x10 grid for 20 steps. Circles mark starting positions and X markers show final positions.

```
RANDOM WALK SIMULATION USING AGENTPY
Inputs: agents=5, grid_size=10, steps=20, seed=42

FINAL-POSITION ANALYSIS
Agent 1: final=(2, 5), successful moves=19/20, unique cells visited=11
Agent 2: final=(3, 0), successful moves=19/20, unique cells visited=12
Agent 3: final=(1, 6), successful moves=19/20, unique cells visited=7
Agent 4: final=(3, 4), successful moves=17/20, unique cells visited=13
Agent 5: final=(6, 7), successful moves=20/20, unique cells visited=15
Occupied cells: 5 of 100
Largest number of agents in one cell: 1
```

SCENARIO B OUTPUT

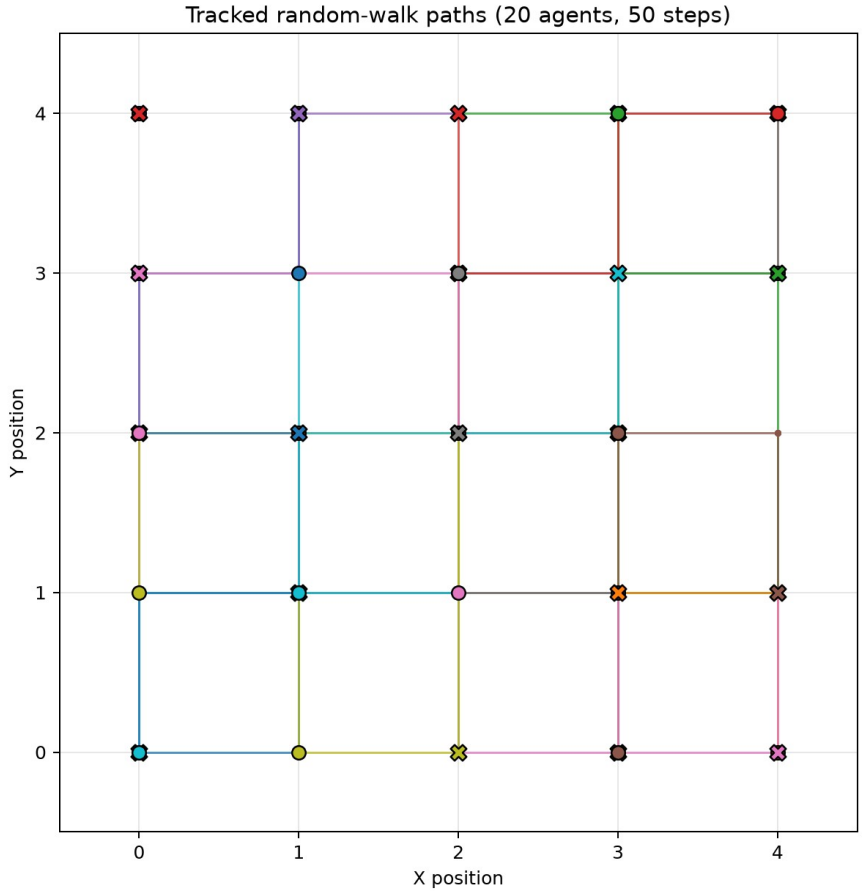


Figure 2. Twenty agents on a 5x5 grid for 50 steps. The dense grid produced many blocked moves.

```
RANDOM WALK SIMULATION USING AGENTPY
Inputs: agents=20, grid_size=5, steps=50, seed=42

FINAL-POSITION ANALYSIS
Agent 1: final=(0, 2), moves=6/50, unique=4    Agent 11: final=(1, 2), moves=11/50, unique=8
Agent 2: final=(2, 3), moves=12/50, unique=8   Agent 12: final=(3, 1), moves=6/50, unique=3
Agent 3: final=(3, 2), moves=8/50, unique=5    Agent 13: final=(4, 3), moves=14/50, unique=5
Agent 4: final=(0, 4), moves=0/50, unique=1    Agent 14: final=(2, 4), moves=10/50, unique=5
Agent 5: final=(0, 0), moves=9/50, unique=5    Agent 15: final=(1, 4), moves=8/50, unique=3
Agent 6: final=(4, 4), moves=11/50, unique=7   Agent 16: final=(4, 1), moves=6/50, unique=6
Agent 7: final=(0, 3), moves=5/50, unique=6    Agent 17: final=(4, 0), moves=11/50, unique=6
Agent 8: final=(3, 4), moves=6/50, unique=5    Agent 18: final=(2, 2), moves=10/50, unique=6
Agent 9: final=(3, 0), moves=6/50, unique=7    Agent 19: final=(2, 0), moves=11/50, unique=5
Agent 10: final=(1, 1), moves=10/50, unique=5  Agent 20: final=(3, 3), moves=10/50, unique=7
Occupied cells: 20 of 25
Largest number of agents in one cell: 1
```

APPENDIX A UPDATED PYTHON CODE

Submitted file: TA1_Panio.py. The code below is the same tested source used to generate both scenarios.

```
"""Random Walk Simulation Using AgentPy.

Student: Alvin Christian S. Panio
Section: AN43-BSCSSE
Professor: Crisola Tan
Date: October 6, 2026

The program records every agent path, prevents agents from entering occupied
cells, summarizes final positions, draws a path plot, and animates the walk.
"""

from collections import Counter
from pathlib import Path

import agentpy as ap
import matplotlib.animation as animation
import matplotlib.pyplot as plt


DIRECTIONS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def positive_integer(prompt):
    """Ask repeatedly until the user enters a positive whole number."""
    while True:
        try:
            value = int(input(prompt))
            if value > 0:
                return value
        except ValueError:
            pass
        print("Please enter a positive whole number.")


class RandomWalker(ap.Agent):
    """An agent that takes one bounded random step at a time."""

    def move(self):
        grid = self.model.grid
        x, y = grid.positions[self]
        dx, dy = self.model.random.choice(DIRECTIONS)
        upper = self.p.grid_size - 1

        # Clamp the intended destination so it always remains inside the grid.
        target = (
            max(0, min(upper, x + dx)),
            max(0, min(upper, y + dy)),
        )

        # Updated movement rule: enter only an unoccupied cell.
        if target in grid.empty:
            grid.move_to(self, target)

        # A blocked or boundary step is recorded as the same position.
        self.path.append(grid.positions[self])


class RandomWalkModel(ap.Model):
    """AgentPy model that owns the agents and their shared grid."""

    def setup(self):
        size = self.p.grid_size
        self.grid = ap.Grid(
            self,
            (size, size),
            torus=False,
            track_empty=True,
```

```
        )
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Start in different random cells so avoidance is valid at time zero.
        self.grid.add_agents(self.agents, random=True, empty=True)
        for agent in self.agents:
            agent.path = [self.grid.positions[agent]]

    def step(self):
        # Randomize update order to avoid giving low-ID agents fixed priority.
        order = list(self.agents)
        self.random.shuffle(order)
        for agent in order:
            agent.move()

    def end(self):
        counts = Counter(self.grid.positions.values())
        self.report("occupied_cells", len(counts))
        self.report("largest_cell_population", max(counts.values(), default=0))


def print_results(model):
    """Print the final positions and path statistics for every agent."""
    print("\nFINAL-POSITION ANALYSIS")
    for number, agent in enumerate(model.agents, start=1):
        path = agent.path
        successful_moves = sum(a != b for a, b in zip(path, path[1:]))
        print(
            f"Agent {number}: final={path[-1]}, "
            f"successful moves={successful_moves}/{model.p.steps}, "
            f"unique cells visited={len(set(path))}"
        )

    print(
        f"Occupied cells: {model.reporters['occupied_cells']} "
        f"of {model.p.grid_size ** 2}"
    )
    print(
        "Largest number of agents in one cell: "
        f"{model.reporters['largest_cell_population']}"
    )


def save_path_plot(model, filename="random_walk_paths.png"):
    """Save a plot of all recorded paths; circles start and X marks finish."""
    fig, ax = plt.subplots(figsize=(8, 7))

    for number, agent in enumerate(model.agents, start=1):
        xs, ys = zip(*agent.path)
        (line,) = ax.plot(
            xs,
            ys,
            marker="o",
            markersize=2.5,
            linewidth=1.2,
            alpha=0.75,
            label=f"Agent {number}",
        )
        ax.scatter(
            xs[0],
            ys[0],
            color=line.get_color(),
            marker="o",
            s=55,
            edgecolors="black",
            zorder=3,
        )
        ax.scatter(
```

```
            xs[-1],
            ys[-1],
            color=line.get_color(),
            marker="X",
            s=75,
            edgecolors="black",
            zorder=3,
        )

    size = model.p.grid_size
    ax.set(
        xlim=(-0.5, size - 0.5),
        ylim=(-0.5, size - 0.5),
        xlabel="X position",
        ylabel="Y position",
        title=(
            f"Tracked random-walk paths ({model.p.agents} agents, "
            f"{model.p.steps} steps)"
        ),
    )
    ax.set_xticks(range(size))
    ax.set_yticks(range(size))
    ax.grid(True, alpha=0.3)
    ax.set_aspect("equal")
    if model.p.agents <= 10:
        ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1), fontsize=8)
    fig.tight_layout()
    output = Path(filename).resolve()
    fig.savefig(output, dpi=180, bbox_inches="tight")
    print(f"Path plot saved to: {output}")
    return fig


def create_animation(model):
    """Build an animation from the already-recorded simulation paths."""
    fig, ax = plt.subplots(figsize=(7, 7))
    size = model.p.grid_size
    ax.set(
        xlim=(-0.5, size - 0.5),
        ylim=(-0.5, size - 0.5),
        xlabel="X position",
        ylabel="Y position",
    )
    ax.set_xticks(range(size))
    ax.set_yticks(range(size))
    ax.grid(True, alpha=0.35)
    ax.set_aspect("equal")

    first_positions = [agent.path[0] for agent in model.agents]
    scatter = ax.scatter(
        [position[0] for position in first_positions],
        [position[1] for position in first_positions],
        c=range(model.p.agents),
        cmap="tab20",
        s=100,
        edgecolors="black",
    )

    def update(frame):
        positions = [agent.path[frame] for agent in model.agents]
        scatter.set_offsets(positions)
        ax.set_title(f"Random walk - step {frame} of {model.p.steps}")
        return (scatter,)

    walk_animation = animation.FuncAnimation(
        fig,
        update,
        frames=model.p.steps + 1,
        interval=250,
        repeat=False,
    )
    return fig, walk_animation
```

```
def main():
    print("RANDOM WALK SIMULATION USING AGENTPY")
    num_agents = positive_integer("Enter number of agents: ")
    grid_size = positive_integer("Enter grid size (e.g., 10 for 10x10): ")
    num_steps = positive_integer("Enter number of steps: ")

    if num_agents > grid_size**2:
        raise SystemExit(
            "Collision avoidance requires no more agents than grid cells."
        )

    parameters = {
        "agents": num_agents,
        "grid_size": grid_size,
        "steps": num_steps,
        "seed": 42,
    }
    model = RandomWalkModel(parameters)
    model.run(display=False)

    print_results(model)
    path_figure = save_path_plot(model)
    animation_figure, walk_animation = create_animation(model)

    # In a headless environment, save the animation instead of opening a GUI.
    if "agg" in plt.get_backend().lower():
        gif_path = Path("random_walk_animation.gif").resolve()
        walk_animation.save(gif_path, writer=animation.PillowWriter(fps=4))
        print(f"Animation saved to: {gif_path}")
        plt.close(path_figure)
        plt.close(animation_figure)
    else:
        plt.show()


if __name__ == "__main__":
    main()
```